

TravelMemory Application Deployment Guide

Project Overview

The TravelMemory application was deployed on AWS using the MERN stack architecture. The deployment consists of separate frontend and backend tiers distributed across multiple Availability Zones for high availability.

Infrastructure Details

VPC: **vpc-0b3f8940a9a457092**

Region: **ap-south-1**

Availability Zones: **ap-south-1b, ap-south-1c**

Frontend Instances: 2 EC2 instances (**TravelMemory-FE-EC2-01, TravelMemory-FE-EC2-02**)

Backend Instances: 2 EC2 instances (**TravelMemory-BE-EC2-01, TravelMemory-BE-EC2-02**)

Frontend ALB: **TravelMemory-FE-ALB-19281157.ap-south-1.elb.amazonaws.com**

Backend ALB: **TravelMemory-BE-ALB-1673317815.ap-south-1.elb.amazonaws.com**

Database: MongoDB Atlas:

MONGO_URI=**mongodb+srv://travelmemoryxxxadmin:xxxxxxxxxxxxx@travelmemory-cluster.xxxxx.mongodb.net/?appName=travelmemory-cluster**

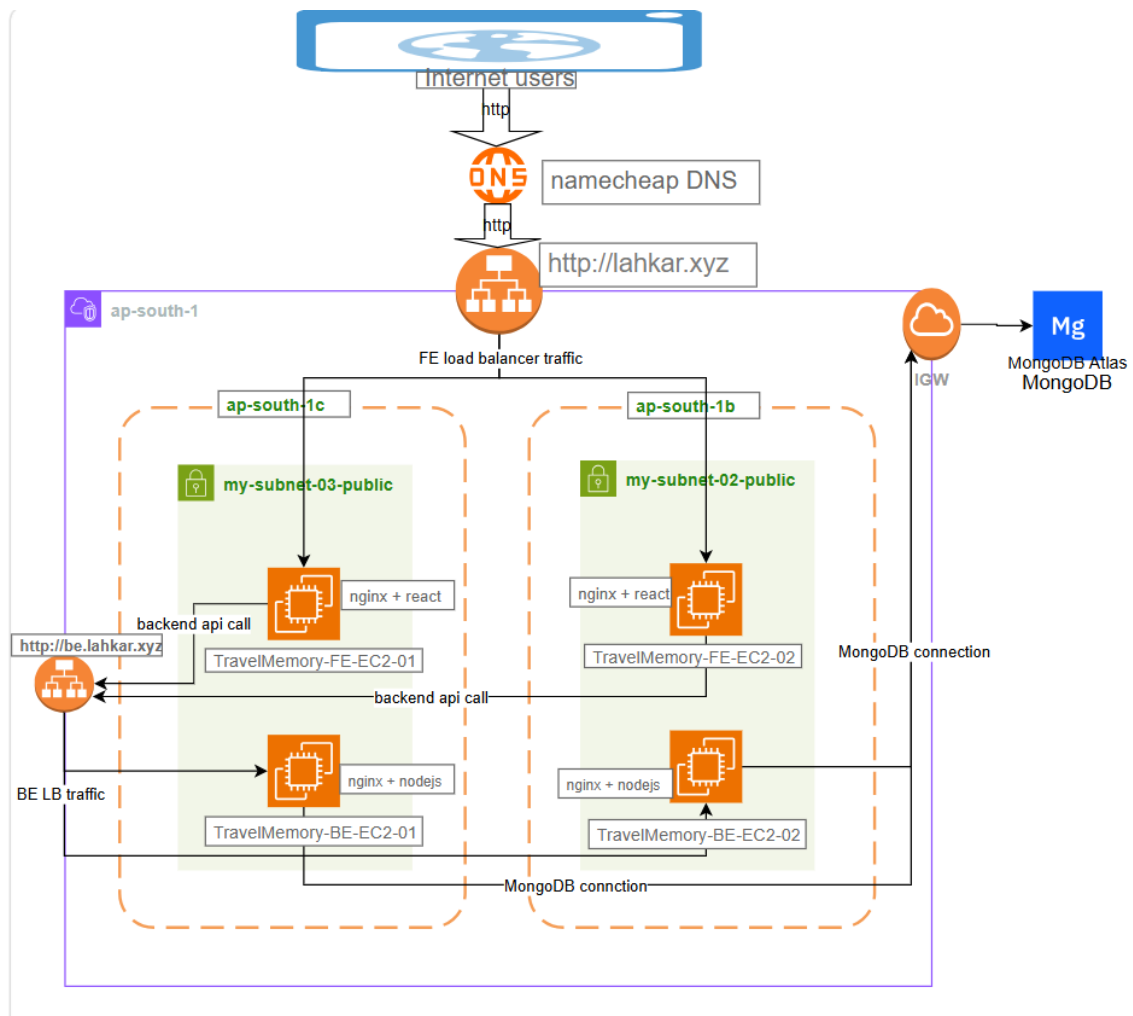
DNS Provider: Namecheap

GitHub Repository: **<https://github.com/AmarGmail/TravelMemory-Deployment>**

The deployment tasks performed via aws cli:

```
hi@USER:~/projects/herovired/TravelMemory/TravelMemory$ echo $AWS_PROFILE
herovired
hi@USER:~/projects/herovired/TravelMemory/TravelMemory$ aws configure list
NAME      : VALUE                                : TYPE      : LOCATION
profile   : herovired                             : env       : ['AWS_PROFILE', 'AWS_DEFAULT_PROFILE']
access_key : *****AN5U                          : shared-credentials-file :
secret_key : *****ayez                          : shared-credentials-file :
region    : ap-south-1                            : config-file  : ~/.aws/config
hi@USER:~/projects/herovired/TravelMemory/TravelMemory$
```

Architecture Diagram



Backend Deployment (ap-south-1c)

Synopsis:

- Launched EC2 instances.
- Configured Node.js backend application.
- Updated backend .env file with MongoDB Atlas connection details.
- Configured Nginx reverse proxy to forward requests to Node.js running on port 3000.
- Verified backend endpoint /hello for health checks.

Details:-

Find the VPC ID and create backend security group

```
hi@USER:~/projects/herovired/aws-credentials$ aws ec2 describe-vpcs --region ap-south-1 --query 'Vpcs[*].Tags[?Key=='Name'']|[0].Value, VpcId]' --output table
+-----+-----+
| DescribeVpcs |
+-----+-----+
| my-vpc-01 | vpc-0b3f8940a9a457092 |
| aws-controltower-VPC | vpc-0b0d4d4cc7d2ee6b4 |
+-----+-----+
hi@USER:~/projects/herovired/aws-credentials$ aws ec2 describe-subnets --filters "Name=vpc-id,Values=vpc-0b3f8940a9a457092" "Name=availability-zone,Values=ap-south-1c" --region ap-south-1 --query 'Subnets[*].SubnetId,AvailabilityZone,CidrBlock,Tags[?Key=='Name'']|[0].Value]' --output table
+-----+-----+-----+-----+
| DescribeSubnets |
+-----+-----+-----+-----+
| subnet-06df2485dc72c6c9 | ap-south-1c | 10.0.31.0/24 | my-subnet-03-public |
| subnet-0e2fe6f48c9458844 | ap-south-1c | 10.0.32.0/24 | my-subnet-03-private |
+-----+-----+-----+-----+
hi@USER:~/projects/herovired/aws-credentials$
hi@USER:~/projects/herovired/aws-credentials$ aws ec2 create-security-group \
--group-name TravelMemory-BE-SG \
--description "Security group for TravelMemory Backend EC2s" \
--vpc-id vpc-0b3f8940a9a457092 \
--region ap-south-1
{
  "GroupId": "sg-0ae92a3ca3ebc23a8",
  "SecurityGroupArn": "arn:aws:ec2:ap-south-1:065194293675:security-group/sg-0ae92a3ca3ebc23a8"
}
hi@USER:~/projects/herovired/aws-credentials$ |
```

Enable ingress/inbound rules for 22(ssh), 80(nginx), and 3000(react app) for the security group created above.

```

hi@USER:~/projects/herovired/aws-credentials$ aws ec2 authorize-security-group-ingress --group-id sg-0ae92a3ca3ebc23a8 \
> --ip-permissions '[
  {
    "IpProtocol": "tcp",
    "FromPort": 22,
    "ToPort": 22,
    "IpRanges": [{"CidrIp": "0.0.0.0/0", "Description": "SSH"}]
  },
  {
    "IpProtocol": "tcp",
    "FromPort": 80,
    "ToPort": 80,
    "IpRanges": [{"CidrIp": "0.0.0.0/0", "Description": "HTTP via Nginx"}]
  },
  {
    "IpProtocol": "tcp",
    "FromPort": 3000,
    "ToPort": 3000,
    "IpRanges": [{"CidrIp": "0.0.0.0/0", "Description": "NodeJS API"}]
  }
] \
--region ap-south-1
{
  "Return": true,
  "SecurityGroupRules": [
    {
      "SecurityGroupRuleId": "sgr-061532add3f052053",
      "GroupId": "sg-0ae92a3ca3ebc23a8",
      "GroupOwnerId": "065194293675",
      "IsEgress": false,
      "IpProtocol": "tcp",
      "FromPort": 22,
      "ToPort": 22,
      "CidrIpv4": "0.0.0.0/0",

```

Launch EC2 using regional ubuntu 24.04 AMI available in the region ap-south-1 with herovired-keypair already exists in the account with the above security group.

```

hi@USER:~/projects/herovired/aws-credentials$ aws ec2 run-instances --image-id ami-006f82a1d5a27da54 \
> --instance-type t3.micro \
> --key-name herovired-keypair \
> --security-group-ids sg-0ae92a3ca3ebc23a8 \
> --subnet-id subnet-06df2485dcb72c6c9 \
> --associate-public-ip-address \
> --tag-specifications 'ResourceType=instance,Tags=[{Key=Name,Value=TravelMemory-BE-EC2-01}]' \
> --region ap-south-1
{
  "ReservationId": "r-0d419ac1cb900f3e0",

```

Clone git source repo.

```

ubuntu@ip-10-0-31-159:~$ pwd
/home/ubuntu
ubuntu@ip-10-0-31-159:~$ sudo git clone https://github.com/UnpredictablePrashant/TravelMemory
Cloning into 'TravelMemory'...
remote: Enumerating objects: 122, done.
remote: Counting objects: 100% (73/73), done.
remote: Compressing objects: 100% (54/54), done.
remote: Total 122 (delta 35), reused 19 (delta 19), pack-reused 49 (from 1)
Receiving objects: 100% (122/122), 199.65 KiB | 6.88 MiB/s, done.
Resolving deltas: 100% (42/42), done.
ubuntu@ip-10-0-31-159:~$

```

Updated backend .env file with MongoDB Atlas connection details.

Install node modules and configured Node.js backend application.

```

node.js v18.20.8
ubuntu@ip-10-0-31-159:~/TravelMemory/backend$ sudo node --version
v18.20.8
ubuntu@ip-10-0-31-159:~/TravelMemory/backend$ ll .env
-rw-r--r-- 1 root root 140 Jun 24 12:56 .env
ubuntu@ip-10-0-31-159:~/TravelMemory/backend$ sudo node index.js
Server started at http://localhost:3000
|

```

Configured Nginx reverse proxy to forward requests to Node.js running on port 3000 same host.

```

ubuntu@ip-10-0-31-159:/etc/nginx/sites-available$ sudo vim custom-server.conf
ubuntu@ip-10-0-31-159:/etc/nginx/sites-available$ cat custom-server.conf
server {
    listen 80;

    location / {
        proxy_pass http://localhost:3000;
    }
}
ubuntu@ip-10-0-31-159:/etc/nginx/sites-available$ sudo ln -s /etc/nginx/sites-available/custom-server.conf
custom-server.conf default
ubuntu@ip-10-0-31-159:/etc/nginx/sites-available$ sudo ln -s /etc/nginx/sites-available/custom-server.conf \
> /etc/nginx/sites-enabled/custom-server.conf
ubuntu@ip-10-0-31-159:/etc/nginx/sites-available$ sudo service nginx configtest
* Testing nginx configuration
ubuntu@ip-10-0-31-159:/etc/nginx/sites-available$ sudo systemctl stop nginx
ubuntu@ip-10-0-31-159:/etc/nginx/sites-available$ sudo systemctl start nginx
ubuntu@ip-10-0-31-159:/etc/nginx/sites-available$ sudo systemctl status nginx
● nginx.service - A high performance web server and a reverse proxy server
   Loaded: loaded (/usr/lib/systemd/system/nginx.service; enabled; preset: enabled)
   Active: active (running) since Wed 2026-06-24 13:22:54 UTC; 7s ago
     Docs: man:nginx(8)
   Process: 9280 ExecStartPre=/usr/sbin/nginx -t -q -g daemon on; master_process on; (code=exited, status=0/SUCCESS)
   Process: 9282 ExecStart=/usr/sbin/nginx -g daemon on; master_process on; (code=exited, status=0/SUCCESS)
   Main PID: 9283 (nginx)
    Tasks: 3 (limit: 1013)
   Memory: 2.6M (peak: 2.8M)
      CPU: 17ms
   CGroup: /system.slice/nginx.service
           └─9283 "nginx: master process /usr/sbin/nginx -g daemon on; master_process on;"
             └─9284 "nginx: worker process"
               └─9285 "nginx: worker process"

Jun 24 13:22:54 ip-10-0-31-159 systemd[1]: Starting nginx.service - A high performance web server and a reverse proxy server...
Jun 24 13:22:54 ip-10-0-31-159 systemd[1]: Started nginx.service - A high performance web server and a reverse proxy server.
ubuntu@ip-10-0-31-159:/etc/nginx/sites-available$ |

```

Verified backend endpoint /hello for health checks.

```

ubuntu@ip-10-0-31-159:/etc/nginx/sites-available$ curl http://localhost
<!DOCTYPE html>
<html lang="en">
<head>
<meta charset="utf-8">
<title>Error</title>
</head>
<body>
<pre>Cannot GET /</pre>
</body>
</html>
ubuntu@ip-10-0-31-159:/etc/nginx/sites-available$ |

```

Frontend Deployment (ap-south-1c)

Synopsis:

- Launched EC2 instances using the Golden Frontend AMI.

- Updated frontend urls.js to point to the Backend ALB endpoint.
- Built React application using npm run build.
- Served static files through Nginx.

Details:-

Create frontend ec2 using regional ubuntu 24.04 AMI in the VPC subnet ap-south-1c with herovired-keypair already exists in the account.

```
hi@USER:~/projects/herovired/aws-credentials$ aws ec2 run-instances --image-id ami-006f82a1d5a27da54 \
> --instance-type t3.micro --key-name herovired-keypair \
> --security-group-ids sg-0ae92a3ca3ebc23a8 --subnet-id subnet-06df2485dcb72c6c9 \
> --associate-public-ip-address \
> --tag-specifications 'ResourceType=instance,Tags=[{Key=Name,Value=TravelMemory-FE-EC2-01}]' \
> --region ap-south-1
{
  "ReservationId": "r-08d4a4a4df377278a",

```

Login to the instance.

```
hi@USER:~/projects/herovired/aws-credentials$ aws ec2 describe-instances \
--filters "Name=tag:Name,Values=TravelMemory-FE-EC2-01" \
--region ap-south-1 \
--query 'Reservations[*].Instances[*].[InstanceId,State.Name,PublicIpAddress,PrivateIpAddress]' \
--output table
-----
| DescribeInstances |
+-----+
| i-01bfc45a7807d46e1 | running | 3.110.211.2 | 10.0.31.108 |
+-----+
hi@USER:~/projects/herovired/aws-credentials$ ssh -i herovired-keypair.pem ubuntu@ 3.110.211.2
ssh: Could not resolve hostname : Name or service not known
hi@USER:~/projects/herovired/aws-credentials$ ssh -i herovired-keypair.pem ubuntu@3.110.211.2
The authenticity of host '3.110.211.2 (3.110.211.2)' can't be established.
ED25519 key fingerprint is SHA256:IeLS8brArnrchsNa9ElREzFZ4zU4roIqfcGSSCjD+8.
This key is not known by any other names.
Are you sure you want to continue connecting (yes/no/[fingerprint])? yes|
```

Clone the source git repo.

Updated frontend urls.js to point to the Backend ALB endpoint.

```
ubuntu@ip-10-0-31-108:~$ sudo git clone https://github.com/UnpredictablePrashant/TravelMemory
Cloning into 'TravelMemory'...
remote: Enumerating objects: 122, done.
remote: Counting objects: 100% (73/73), done.
remote: Compressing objects: 100% (54/54), done.
remote: Total 122 (delta 35), reused 19 (delta 19), pack-reused 49 (from 1)
Receiving objects: 100% (122/122), 199.65 KiB | 9.07 MiB/s, done.
Resolving deltas: 100% (42/42), done.
ubuntu@ip-10-0-31-108:~$ npm --version
10.8.2
ubuntu@ip-10-0-31-108:~$ |
```

Launched EC2 instances using the Golden Frontend AMI.
Served static files through Nginx.

```

ubuntu@ip-10-0-31-108:~$ sudo unlink /etc/nginx/sites-enabled/default
ubuntu@ip-10-0-31-108:~$ cd /etc/nginx/sites-available/
ubuntu@ip-10-0-31-108:/etc/nginx/sites-available$ sudo vim custom_server.conf
ubuntu@ip-10-0-31-108:/etc/nginx/sites-available$ cat custom_server.conf
server {
    listen 80;
    location / {
        proxy_pass http://localhost:3000;
    }
}
ubuntu@ip-10-0-31-108:/etc/nginx/sites-available$ sudo ln -s /etc/nginx/sites-available/custom_server.conf /etc/nginx/sites-enabled/custom_server.conf
ubuntu@ip-10-0-31-108:/etc/nginx/sites-available$ sudo systemctl nginx configtest
Unknown command verb 'nginx', did you mean 'link'?
ubuntu@ip-10-0-31-108:/etc/nginx/sites-available$ sudo nginx configtest
nginx: invalid option: "configtest"
ubuntu@ip-10-0-31-108:/etc/nginx/sites-available$ sudo nginx -t
nginx: the configuration file /etc/nginx/nginx.conf syntax is ok
nginx: configuration file /etc/nginx/nginx.conf test is successful
ubuntu@ip-10-0-31-108:/etc/nginx/sites-available$ sudo systemctl restart nginx
ubuntu@ip-10-0-31-108:/etc/nginx/sites-available$ sudo systemctl enable nginx
Synchronizing state of nginx.service with SysV service script with /usr/lib/systemd/systemd-sysv-install.
Executing: /usr/lib/systemd/systemd-sysv-install enable nginx
ubuntu@ip-10-0-31-108:/etc/nginx/sites-available$

```

Load Golden Images and replicate EC2 in ap-south-1b

Create golden images for backend and frontend from the above two instances created in zone ap-south-1c and note down the golden image ids.

```

(venv) hi@USER:~/projects/herovired$ aws ec2 describe-instances --filters "Name=tag:Name,Values=TravelMemory-BE-EC2-01" --region ap-south-1 --query 'Reservations[*].Instances[*].[InstanceId,PrivateIpAddress,State.Name]' --output table
DescribeInstances
+-----+-----+-----+
| i-0c9e618965a470982 | 10.0.31.159 | running |
+-----+-----+-----+
(venv) hi@USER:~/projects/herovired$ aws ec2 describe-instances --filters "Name=tag:Name,Values=TravelMemory-FE-EC2-01" --region ap-south-1 --query 'Reservations[*].Instances[*].[InstanceId,PrivateIpAddress,State.Name]' --output table
DescribeInstances
+-----+-----+-----+
| i-01bfc45a7807d46e1 | 10.0.31.108 | running |
+-----+-----+-----+
(venv) hi@USER:~/projects/herovired$ aws ec2 create-image \
--instance-id i-0c9e618965a470982 \
--name "TravelMemory-BE-Golden-AMI-v1" \
--description "Golden AMI for TravelMemory Backend" \
--no-reboot \
--region ap-south-1
{
  "ImageId": "ami-00194d70797df5128"
}
(venv) hi@USER:~/projects/herovired$ aws ec2 create-image \
--instance-id i-01bfc45a7807d46e1 \
--name "TravelMemory-FE-Golden-AMI-v1" \
--description "Golden AMI for TravelMemory Frontend" \
--no-reboot \
--region ap-south-1
{
  "ImageId": "ami-0db10a9ac011bd019"
}
(venv) hi@USER:~/projects/herovired$

```

Golden Images status-

```
(venv) hi@USER:~/projects/herovired$ aws ec2 describe-images \
--owners self \
--filters "Name=name,Values=TravelMemory-BE-Golden-AMI-v1" \
--query 'Images[*].[ImageId,Name,State]' \
--output table \
--region ap-south-1
```

DescribeImages		
ami-00194d70797df5128	TravelMemory-BE-Golden-AMI-v1	available

```
(venv) hi@USER:~/projects/herovired$ aws ec2 describe-images \
--owners self \
--filters "Name=name,Values=TravelMemory-FE-Golden-AMI-v1" \
--query 'Images[*].[ImageId,Name,State]' \
--output table \
--region ap-south-1
```

DescribeImages		
ami-0db10a9ac011bd019	TravelMemory-FE-Golden-AMI-v1	available

```
(venv) hi@USER:~/projects/herovired$ aws ec2 describe-subnets \
--filters \
"Name=vpc-id,Values=vpc-0b3f8940a9a457092" \
"Name=availability-zone,Values=ap-south-1b" \
--region ap-south-1 \
--query 'Subnets[*].[SubnetId,AvailabilityZone,CidrBlock,Tags[?Key==`Name`][0].Value]' \
--output table
```

DescribeSubnets			
subnet-056aba037859eb236	ap-south-1b	10.0.12.0/24	my-subnet-02-private
subnet-066558ba2f3031449	ap-south-1b	10.0.11.0/24	my-subnet-02-public

```
(venv) hi@USER:~/projects/herovired$ aws ec2 run-instances \
--image-id ami-00194d70797df5128 \
^C
```

Launched backend EC2 instances using the Golden Backend AMI in ap-south-1b in the public subnet.

```
(venv) hi@USER:~/projects/herovired$ aws ec2 run-instances \
--image-id ami-00194d70797df5128 \
--instance-type t3.micro \
--key-name herovired-keypair \
--security-group-ids sg-0ae92a3ca3ebc23a8 \
--subnet-id subnet-066558ba2f3031449 \
--associate-public-ip-address \
--tag-specifications \
'ResourceType=instance,Tags=[{Key=Name,Value=TravelMemory-BE-EC2-02}]' \
--region ap-south-1
```

```
{
  "ReservationId": "r-02e7553c7dc7321f7",
}
```

Launch frontend EC2 using the Golden Frontend AMI in ap-south-1b in public subnet.

```
(venv) hi@USER:~/projects/herovired$ aws ec2 run-instances \
--image-id ami-0db10a9ac011bd019 \
--instance-type t3.micro \
--key-name herovired-keypair \
--security-group-ids sg-0c092cfe1388d5a7d \
--subnet-id subnet-066558ba2f3031449 \
--associate-public-ip-address \
--tag-specifications \
'ResourceType=instance,Tags=[{Key=Name,Value=TravelMemory-FE-EC2-02}]' \
--region ap-south-1
```

```
{
  "ReservationId": "r-082984923d86366ae",
}
```

The FE and BE instances status after the deployment -

```
(venv) hi@USER:~/projects/herovired$ aws ec2 describe-instances --filters "Name=tag:Name,Values=TravelMemory-*" --query 'Reservations[*].Instances[*].Tags[?Key==`Name`][0].Value,InstanceId,Placement.AvailabilityZone,State.Name,PrivateIpAddress,PublicIpAddress' --output table --region ap-south-1
```

DescribeInstances						
TravelMemory-BE-EC2-01	i-0c9e618965a470982	ap-south-1c	running	10.0.31.159	3.111.56.70	
TravelMemory-FE-EC2-01	i-01bfc45a7887d46e1	ap-south-1c	running	10.0.31.108	3.110.211.2	
TravelMemory-BE-EC2-02	i-0136d312f560309d3	ap-south-1b	running	10.0.11.74	13.203.202.230	
TravelMemory-FE-EC2-02	i-08aefbe6499c1d233	ap-south-1b	running	10.0.11.180	15.206.195.64	

```
(venv) hi@USER:~/projects/herovired$
```

Load Balancing and High Availability

Synopsis:

- Created separate target groups for frontend and backend.
- Registered EC2 instances in respective target groups.
- Created separate Application Load Balancers.
- Configured health checks.
- Distributed traffic across multiple Availability Zones.

Details:-

Create two security groups one each for frontend ALB and backend ALB.

```
hi@USER:~/projects/herovired/aws-credentials$ aws ec2 describe-instances \
--filters "Name=tag:Name,Values=TravelMemory-BE-EC2-*" \
--region ap-south-1 \
--query 'Reservations[*].Instances[*].Tags[?Key==`Name`][0].Value,InstanceId,PrivateIpAddress' \
--output table
```

DescribeInstances		
TravelMemory-BE-EC2-01	i-0c9e618965a470982	10.0.31.159
TravelMemory-BE-EC2-02	i-0136d312f560309d3	10.0.11.74

```
hi@USER:~/projects/herovired/aws-credentials$ aws ec2 describe-instances \
--filters "Name=tag:Name,Values=TravelMemory-FE-EC2-*" \
--region ap-south-1 \
--query 'Reservations[*].Instances[*].Tags[?Key==`Name`][0].Value,InstanceId,PrivateIpAddress' \
--output table
```

DescribeInstances		
TravelMemory-BE-EC2-01	i-0c9e618965a470982	10.0.31.159
TravelMemory-BE-EC2-02	i-0136d312f560309d3	10.0.11.74

```
hi@USER:~/projects/herovired/aws-credentials$ aws ec2 create-security-group --group-name "TravelMemory-FE-ALB-SG" --description "ALB SG for frontend app" --vpc-id vpc-0b3f8940a9a457092 --region ap-south-1
{
  "GroupId": "sg-0ee12984862c6c56c",
  "SecurityGroupArn": "arn:aws:ec2:ap-south-1:065194293675:security-group/sg-0ee12984862c6c56c"
}
hi@USER:~/projects/herovired/aws-credentials$ aws ec2 create-security-group --group-name "TravelMemory-BE-ALB-SG" --description "ALB SG for backend app" --vpc-id vpc-0b3f8940a9a457092 --region ap-south-1
{
  "GroupId": "sg-0a44b5af3baf945d4",
  "SecurityGroupArn": "arn:aws:ec2:ap-south-1:065194293675:security-group/sg-0a44b5af3baf945d4"
}
hi@USER:~/projects/herovired/aws-credentials$
```

Note down the security group ids above. Create inbound rules for port 80 for both the ALB SGs.

```

hi@USER:~/projects/herovired/aws-credentials$ aws ec2 authorize-security-group-ingress --group-id sg-0ee12984862c6c56c --protocol tcp --port 80 --cidr 0.0.0.0/0 --region ap-south-1
{
  "Return": true,
  "SecurityGroupRules": [
    {
      "SecurityGroupRuleId": "sgr-03bb4513834b8f49d",
      "GroupId": "sg-0ee12984862c6c56c",
      "GroupOwnerId": "065194293675",
      "IsEgress": false,
      "IpProtocol": "tcp",
      "FromPort": 80,
      "ToPort": 80,
      "CidrIpv4": "0.0.0.0/0",
      "SecurityGroupRuleArn": "arn:aws:ec2:ap-south-1:065194293675:security-group-rule/sgr-03bb4513834b8f49d"
    }
  ]
}
hi@USER:~/projects/herovired/aws-credentials$ aws ec2 authorize-security-group-ingress --group-id sg-0a44b5af3baf945d4 --protocol tcp --port 80 --cidr 0.0.0.0/0 --region ap-south-1
{
  "Return": true,
  "SecurityGroupRules": [
    {
      "SecurityGroupRuleId": "sgr-00572e8a905ed6d9d",
      "GroupId": "sg-0a44b5af3baf945d4",
      "GroupOwnerId": "065194293675",
      "IsEgress": false,
      "IpProtocol": "tcp",
      "FromPort": 80,
      "ToPort": 80,
      "CidrIpv4": "0.0.0.0/0",
      "SecurityGroupRuleArn": "arn:aws:ec2:ap-south-1:065194293675:security-group-rule/sgr-00572e8a905ed6d9d"
    }
  ]
}
hi@USER:~/projects/herovired/aws-credentials$

```

Create backend target group for backend load balancer and register the backend EC2 instances.

```

hi@USER:~/projects/herovired/aws-credentials$ aws elbv2 create-target-group --name TravelMemory-BE-TG --protocol HTTP --port 80 --target-type instance --vpc-id vpc-0b3f8940a9a457092 --health-check-protocol HTTP --health-check-port traffic-port --health-check-path / --region ap-south-1
{
  "TargetGroups": [
    {
      "TargetGroupArn": "arn:aws:elasticloadbalancing:ap-south-1:065194293675:targetgroup/TravelMemory-BE-TG/23d02b8ab686ddd9",
      "TargetGroupName": "TravelMemory-BE-TG",
      "Protocol": "HTTP",
      "Port": 80,
      "VpcId": "vpc-0b3f8940a9a457092",
      "HealthCheckProtocol": "HTTP",
      "HealthCheckPort": "traffic-port",
      "HealthCheckEnabled": true,
      "HealthCheckIntervalSeconds": 30,
      "HealthCheckTimeoutSeconds": 5,
      "HealthyThresholdCount": 5,
      "UnhealthyThresholdCount": 2,
      "HealthCheckPath": "/",
      "Matcher": {
        "HttpCode": "200"
      },
      "TargetType": "instance",
      "ProtocolVersion": "HTTP1",
      "IpAddressType": "ipv4"
    }
  ]
}
hi@USER:~/projects/herovired/aws-credentials$ aws elbv2 register-targets --target-group-arn arn:aws:elasticloadbalancing:ap-south-1:065194293675:targetgroup/TravelMemory-BE-TG/23d02b8ab686ddd9 --targets Id=i-0c9e618965a470982 Id=i-0136d312f560309d3 --region=ap-south-1
hi@USER:~/projects/herovired/aws-credentials$

```

Create frontend target group for frontend load balancer and register the frontend EC2 instances.

```

hi@USER:~/projects/herovired/aws-credentials$ aws elbv2 create-target-group --name TravelMemory-FE-TG --protocol HTTP --port 80 --vpc-id vpc-0b3f8940a9a457092 --target-type instance --health-check-protocol HTTP --health-check-path "/" --health-check-port traffic-port --match HttpStatusCode=200
{
  "TargetGroups": [
    {
      "TargetGroupArn": "arn:aws:elasticloadbalancing:ap-south-1:065194293675:targetgroup/TravelMemory-FE-TG/efac7cf2807d4599",
      "TargetGroupName": "TravelMemory-FE-TG",
      "Protocol": "HTTP",
      "Port": 80,
      "VpcId": "vpc-0b3f8940a9a457092",
      "HealthCheckProtocol": "HTTP",
      "HealthCheckPort": "traffic-port",
      "HealthCheckEnabled": true,
      "HealthCheckIntervalSeconds": 30,
      "HealthCheckTimeoutSeconds": 5,
      "HealthyThresholdCount": 5,
      "UnhealthyThresholdCount": 2,
      "HealthCheckPath": "/",
      "Matcher": {
        "HttpCode": "200"
      },
      "TargetType": "instance",
      "ProtocolVersion": "HTTP1",
      "IpAddressType": "ipv4"
    }
  ]
}
hi@USER:~/projects/herovired/aws-credentials$ aws elbv2 register-targets --target-arn arn:aws:elasticloadbalancing:ap-south-1:065194293675:targetgroup/TravelMemory-FE-TG/efac7cf2807d4599 --targets Id=i-01bfc45a7807d46e1 Id=i-08aefbe6499c1d233
aws: [ERROR]: An error occurred (ParamValidation): the following arguments are required: --target-group-arn
usage: aws [options] <command> [<subcommand> ...] [parameters]
To see help text, you can run:

aws help
aws <command> help
aws <command> <subcommand> help
hi@USER:~/projects/herovired/aws-credentials$ aws elbv2 register-targets --target-group-arn arn:aws:elasticloadbalancing:ap-south-1:065194293675:targetgroup/TravelMemory-FE-TG/efac7cf2807d4599 --targets Id=i-01bfc45a7807d46e1 Id=i-08aefbe6499c1d233
hi@USER:~/projects/herovired/aws-credentials$

```

Create backend load balancer and note down the arn.

```

hi@USER:~/projects/herovired/aws-credentials$ aws elbv2 create-load-balancer --name TravelMemory-BE-ALB --subnets subnet-96df2485dcb72c6c9 subnet-966558ba2f3031449 --security-groups sg-0a44b5af3baf945d4 --scheme internet-facing --type application --ip-address-type ipv4
{
  "LoadBalancers": [
    {
      "LoadBalancerArn": "arn:aws:elasticloadbalancing:ap-south-1:065194293675:loadbalancer/app/TravelMemory-BE-ALB/5ded73ea27d84eac",
      "DNSName": "TravelMemory-BE-ALB-1673317815.ap-south-1.elb.amazonaws.com",
      "CanonicalHostedZoneId": "ZP97RAFLXTN2K",
      "CreatedTime": "2026-06-25T05:46:31.079000+00:00",
      "LoadBalancerName": "TravelMemory-BE-ALB",
      "Scheme": "internet-facing",
      "VpcId": "vpc-0b3f8940a9a457092",
      "State": {
        "Code": "provisioning"
      },
      "Type": "application",
      "AvailabilityZones": [
        {
          "ZoneName": "ap-south-1c",
          "SubnetId": "subnet-96df2485dcb72c6c9",
          "LoadBalancerAddresses": []
        },
        {
          "ZoneName": "ap-south-1b",
          "SubnetId": "subnet-966558ba2f3031449",
          "LoadBalancerAddresses": []
        }
      ],
      "SecurityGroups": [
        "sg-0a44b5af3baf945d4"
      ],
      "IpAddressType": "ipv4"
    }
  ]
}

```

Similarly, create frontend ALB

```

h1@USER:~/projects/herovired/aws-credentials$ aws elbv2 create-load-balancer --name TravelMemory-FE-ALB --subnets subnet-06df2485dc72c6c9 subnet-066558ba2f3031449 --security-groups sg-0ee12984862c6c56c --scheme internet-facing --type application --ip-address-type ipv4
{
  "LoadBalancers": [
    {
      "LoadBalancerArn": "arn:aws:elasticloadbalancing:ap-south-1:065194293675:loadbalancer/app/TravelMemory-FE-ALB/33028d5ce35ab248",
      "DNSName": "TravelMemory-FE-ALB-19281157.ap-south-1.elb.amazonaws.com",
      "CanonicalHostedZoneId": "ZP97RAFLXTNZK",
      "CreatedTime": "2026-06-25T05:35:05.750000+00:00",
      "LoadBalancerName": "TravelMemory-FE-ALB",
      "Scheme": "internet-facing",
      "VpcId": "vpc-0b3f8940a9a457092",
      "State": {
        "Code": "provisioning"
      },
      "Type": "application",
      "AvailabilityZones": [
        {
          "ZoneName": "ap-south-1c",
          "SubnetId": "subnet-06df2485dc72c6c9",
          "LoadBalancerAddresses": []
        },
        {
          "ZoneName": "ap-south-1b",
          "SubnetId": "subnet-066558ba2f3031449",
          "LoadBalancerAddresses": []
        }
      ],
      "SecurityGroups": [
        "sg-0ee12984862c6c56c"
      ],
      "IpAddressType": "ipv4"
    }
  ]
}

```

Create backend listener

```

h1@USER:~/projects/herovired/aws-credentials$ aws elbv2 create-listener --load-balancer-arn arn:aws:elasticloadbalancing:ap-south-1:065194293675:loadbalancer/app/TravelMemory-BE-ALB/5ded73ea27d84eac --protocol HTTP --port 80 --default-actions Type=forward,TargetGroupArn=arn:aws:elasticloadbalancing:ap-south-1:065194293675:targetgroup/TravelMemory-BE-TG/23d02b8ab686ddd9
{
  "Listeners": [
    {
      "ListenerArn": "arn:aws:elasticloadbalancing:ap-south-1:065194293675:listener/app/TravelMemory-BE-ALB/5ded73ea27d84eac/f047eac8de0c0a4c",
      "LoadBalancerArn": "arn:aws:elasticloadbalancing:ap-south-1:065194293675:loadbalancer/app/TravelMemory-BE-ALB/5ded73ea27d84eac",
      "Port": 80,
      "Protocol": "HTTP",
      "DefaultActions": [
        {
          "Type": "forward",
          "TargetGroupArn": "arn:aws:elasticloadbalancing:ap-south-1:065194293675:targetgroup/TravelMemory-BE-TG/23d02b8ab686ddd9",
          "ForwardConfig": {
            "TargetGroups": [
              {
                "TargetGroupArn": "arn:aws:elasticloadbalancing:ap-south-1:065194293675:targetgroup/TravelMemory-BE-TG/23d02b8ab686ddd9",
                "Weight": 1
              }
            ],
            "TargetGroupStickinessConfig": {
              "Enabled": false
            }
          }
        }
      ]
    }
  ]
}

```

Create frontend listener

```

h1@USER:~/projects/herovired/aws-credentials$ aws elbv2 create-listener --load-balancer-arn arn:aws:elasticloadbalancing:ap-south-1:065194293675:loadbalancer/app/TravelMemory-FE-ALB/33028d5ce35ab248 --protocol HTTP --port 80 --default-actions Type=forward,TargetGroupArn=arn:aws:elasticloadbalancing:ap-south-1:065194293675:targetgroup/TravelMemory-FE-TG/efac7cf2807d4599
{
  "Listeners": [
    {
      "ListenerArn": "arn:aws:elasticloadbalancing:ap-south-1:065194293675:listener/app/TravelMemory-FE-ALB/33028d5ce35ab248/bc0cd6a6fad0f895",
      "LoadBalancerArn": "arn:aws:elasticloadbalancing:ap-south-1:065194293675:loadbalancer/app/TravelMemory-FE-ALB/33028d5ce35ab248",
      "Port": 80,
      "Protocol": "HTTP",
      "DefaultActions": [
        {
          "Type": "forward",
          "TargetGroupArn": "arn:aws:elasticloadbalancing:ap-south-1:065194293675:targetgroup/TravelMemory-FE-TG/efac7cf2807d4599",
          "ForwardConfig": {
            "TargetGroups": [
              {
                "TargetGroupArn": "arn:aws:elasticloadbalancing:ap-south-1:065194293675:targetgroup/TravelMemory-FE-TG/efac7cf2807d4599",
                "Weight": 1
              }
            ],
            "TargetGroupStickinessConfig": {
              "Enabled": false
            }
          }
        }
      ]
    }
  ]
}

```

Check ALB status-

```
hi@USER:~/projects/herovired/aws-credentials$ aws elbv2 describe-load-balancers \
--query 'LoadBalancers[*].[LoadBalancerName,DNSName,State.Code]' \
--output table
```

DescribeLoadBalancers		
TravelMemory-FE-ALB	TravelMemory-FE-ALB-19281157.ap-south-1.elb.amazonaws.com	active
TravelMemory-BE-ALB	TravelMemory-BE-ALB-1673317815.ap-south-1.elb.amazonaws.com	active

```
hi@USER:~/projects/herovired/aws-credentials$
```

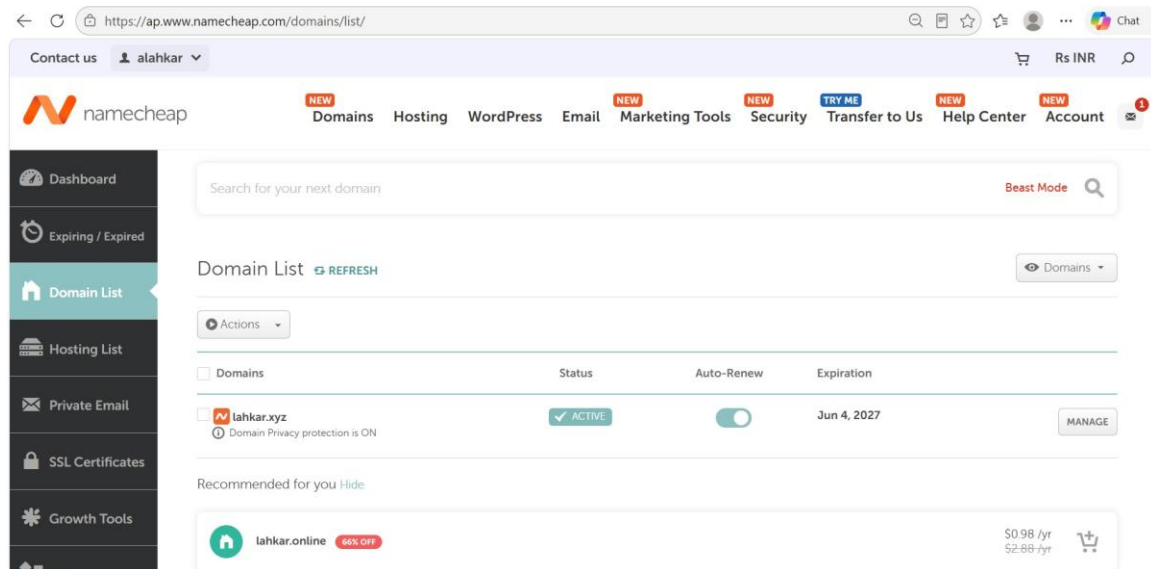
This would distribute the traffic across multiple Availability Zones.

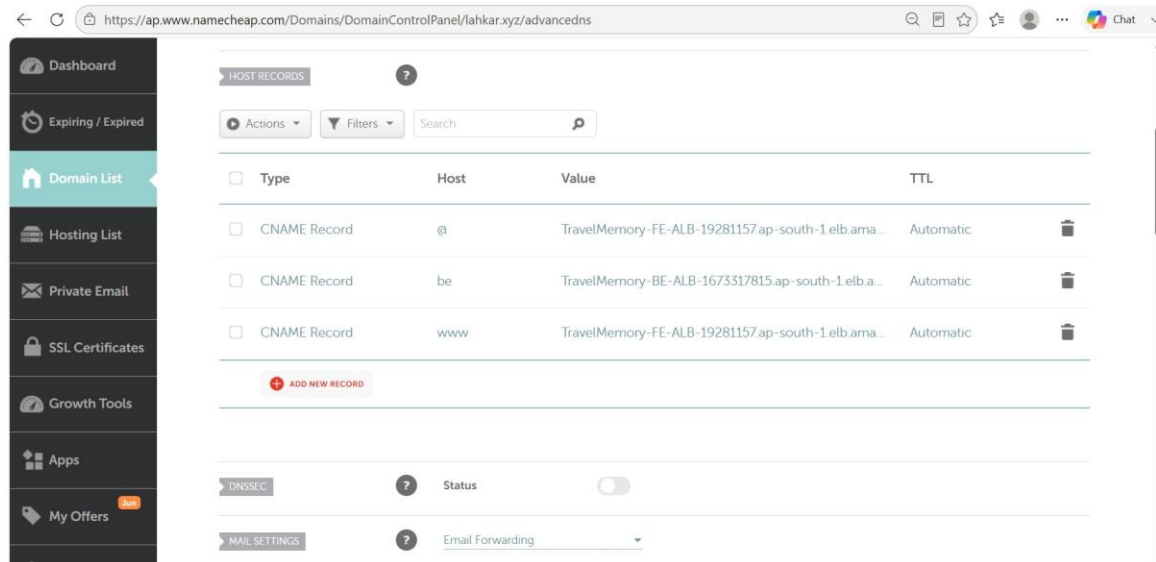
DNS Configuration

Created CNAME records in Namecheap:

www -> Frontend ALB DNS

be -> Backend ALB DNS





Verify DNS records are updated expectedly.

```
hi@USER:~/projects/herovired/aws-credentials$ host lahkar.xyz
lahkar.xyz is an alias for travelmemory-fe-alb-19281157.ap-south-1.elb.amazonaws.com.
travelmemory-fe-alb-19281157.ap-south-1.elb.amazonaws.com has address 13.232.158.239
travelmemory-fe-alb-19281157.ap-south-1.elb.amazonaws.com has address 13.207.211.153
hi@USER:~/projects/herovired/aws-credentials$ host be.lahkar.xyz
be.lahkar.xyz is an alias for travelmemory-be-alb-1673317815.ap-south-1.elb.amazonaws.com.
travelmemory-be-alb-1673317815.ap-south-1.elb.amazonaws.com has address 13.126.57.166
travelmemory-be-alb-1673317815.ap-south-1.elb.amazonaws.com has address 13.234.50.191
hi@USER:~/projects/herovired/aws-credentials$
```

Security Best Practices

Used dedicated Security Groups for frontend, backend, and ALBs.

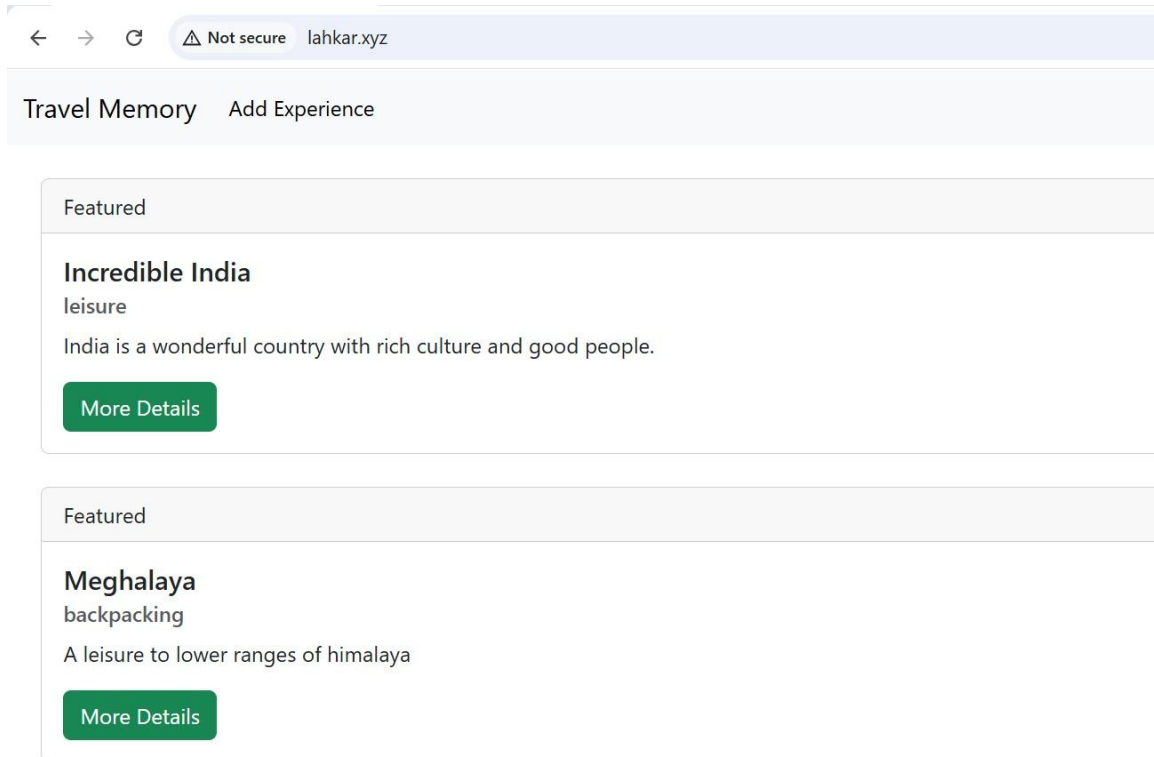
Restricted backend access through Backend ALB.

Excluded .env and node_modules from Git repository

(<https://github.com/AmarGmail/TravelMemory-Deployment>)

Verification

Verified frontend accessibility through Frontend ALB.



Verified backend health endpoint through Backend ALB.
Confirmed application functionality end-to-end.

Architecture Diagram

Refer to the attached draw.io architecture diagram included in the repository and also at the start of this document.

GitHub Repository

Only the source code (no node_modules)

Frontend/.env removed as this include mongodb secrets.

<https://github.com/AmarGmail/TravelMemory-Deployment>